Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
Informatik und
Wirtschaftsinformatik

Laborbericht

Optimierung eines Raytracers

im Studiengang
Informatik Master

vorgelegt von

Florian Kaiser
Matrikelnummer: 79728
kafl1018@h-ka.de

an der Hochschule Karlsruhe
am 06.11.2026

Prüfer: Prof. Dr. Christian Pape

1 Einleitung

Ein Raytracer berechnet ein Bild, indem er durch jedes Pixel einen Sehstrahl von der Kamera in die Szene schickt und das erste Objekt sucht, das dieser Strahl trifft. Im Trefferpunkt wird die Helligkeit bestimmt. Ob die Lichtquelle von dort aus sichtbar ist, prüft ein weiterer Strahl, der Schattenstrahl. Beim Verfahren nach Whitted [1] werden an spiegelnden und durchsichtigen Oberflächen zudem rekursiv ein reflektierter und ein gebrochener Strahl verfolgt. Fast die gesamte Rechenzeit entfällt dabei auf die Frage, welches Dreieck ein Strahl als erstes trifft. An dieser Stelle setzen alle Aufgaben der Rechnerübung an.

Gemessen wird ausschließlich mit der vorgegebenen Testszene `teapot_n_glass.obj`. Sie zeigt eine Teekanne, drei Töpfe aus Glas, Porzellan und Ton sowie eine runde Bodenplatte aus zusammen 18 032 Dreiecken. In der ersten Aufgabe wird jeder Strahl gegen alle Dreiecke geprüft. Die zweite ersetzt diese Suche durch einen k-d-Baum und die dritte verfolgt mehrere Strahlen gemeinsam mit AVX-Befehlen. Jede Aufgabe wird im Folgenden in einem eigenen Kapitel mit Entwurf, Bildergebnissen und Zeitmessungen beschrieben.

1.1 Messmethodik

Damit die Zeiten aller Aufgaben vergleichbar bleiben, legt das Programm Kamera und Lichtquelle allein aus der Szene fest (Abschnitt 2.1). Die Zeiten misst das Programm selbst mit `std::chrono::steady_clock`, getrennt für das Einlesen der Szene und das Rendern. Zusätzlich zählt es die Strahlen jeder Art und die Schnitttests. Das Skript `messung/messen.sh` baut den Raytracer im Release-Modus mit `-O3`, startet einen Aufwärmlauf und misst danach jede Konfiguration dreimal. Jeder Lauf landet als Zeile in einer CSV-Datei, aus der die Tabellen beim Setzen des Berichts berechnet werden. Angegeben ist, wenn nicht anders vermerkt, der Median der drei Läufe. Gerendert wird in einem einzigen Thread. Da `rand()` ohne eigenen Startwert läuft, erzeugt jeder Lauf einer Konfiguration dasselbe Bild und dieselben Zählerstände. Gemessen wurde auf einem MacBook Pro mit Apple M1 und 16 GB Arbeitsspeicher unter macOS 26.5, übersetzt mit Apple Clang 21.0.0.

Für Teile des Programmcodes, die Unit-Tests und das Messskript wurde Claude (Anthropic) eingesetzt. Code, Messwerte wurden vom Autor geprüft.

2 Aufgabe 1: Brute-Force-Raytracer

In der ersten Aufgabe entsteht ein Whitted-Raytracer ohne Beschleunigungsstruktur, der jeden Strahl gegen alle Dreiecke der Szene testet. Er dient als Referenz für die folgenden Aufgaben. Im Mittelpunkt stehen die beiden Schnitttests zwischen Strahl und Dreieck nach Badouel und nach Möller und Trumbore, die über die Option `--badouel` gegeneinander ausgetauscht werden.

2.1 Programmentwurf

Grundlage ist die Quelltextbasis aus dem Computergrafiklabor. Das Programm ist in C++20 geschrieben, wird mit CMake gebaut und braucht außer GoogleTest keine Bibliotheken. Der Quelltext liegt in fünf Verzeichnissen. `math/` enthält Vektoren und Farben und `geometry/` Strahl, Kugel und Dreieck aus dem Labor. `scene/` fasst Objekte, Materialien und Lichtquellen in der Klasse `Scene` zusammen, `render/` berechnet daraus das Bild und `io/` enthält den Wavefront-Import und die BMP-Ausgabe. Abbildung 1 zeigt die Klassen der drei oberen Verzeichnisse. Jede Klasse hat dabei eine Aufgabe. Der `Renderer` läuft über alle Pixel, die `Camera` erzeugt den Sehstrahl zu einer Stelle im Bild und der `Shader` berechnet die Farbe entlang eines Strahls. In einer früheren Fassung hieß die Pixelschleife `Window`, obwohl das Programm kein Fenster öffnet, und die Kamera berechnete auch die Beleuchtung.

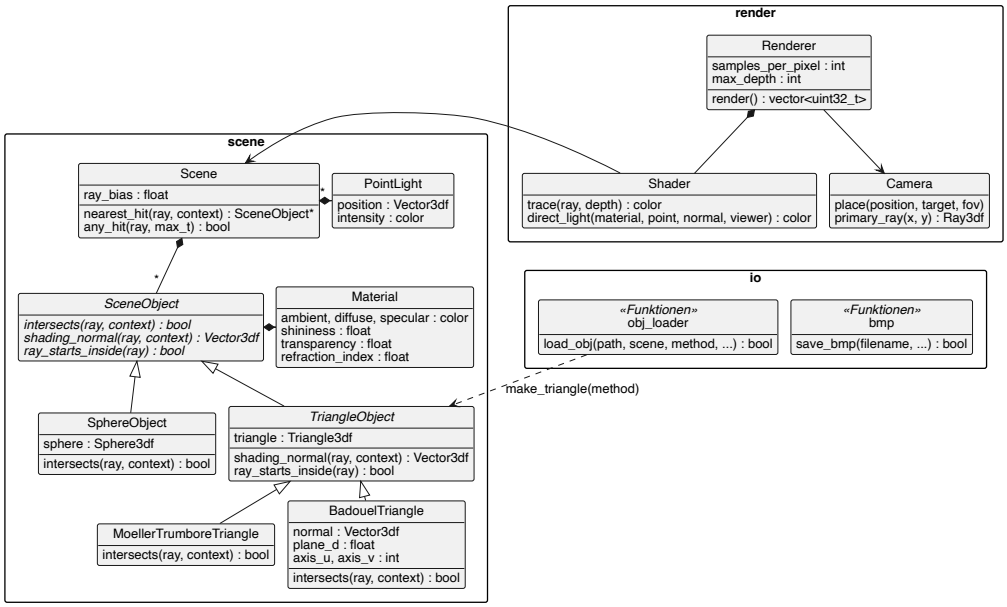


Abbildung 1: Klassendiagramm des Raytracers (`uml/klassen.puml`)

Ein Objekt der Szene ist eine Unterklasse von `SceneObject`. Die abstrakte Methode `intersects` liefert den Schnitt mit einem Strahl, `shading_normal` die Normale für die Beleuchtung und `ray_starts_inside` die Richtung der Brechung. Die beiden Schnittverfahren sind eigene Unterklassen des Dreiecks und überschreiben nur `intersects`. Welches Verfahren gilt, entscheidet der Import einmal beim Anlegen der Dreiecke. Die Suchen der Szene rufen danach nur noch `intersects` auf und kennen das Verfahren nicht. In einer früheren Fassung fragten beide Suchen bei jedem einzelnen Test ab, welches Verfahren eingestellt ist.

Die Szene hält die Objekte als Zeiger in einem `std::vector`. Ein Dreieck nach Möller-Trumbore belegt 128 Byte, davon 72 Byte für drei Eckpunkte und drei Eckpunktnormalen, 48 Byte für das Material und 8 Byte für den Zeiger auf die Tabelle der virtuellen Methoden. Das Dreieck nach Badouel speichert zusätzlich seine Ebene und belegt 152 Byte. Die rund 2,5 MB der Testszene werden bei der Brute-Force-Suche für jeden Strahl einmal vollständig durchlaufen.

Die Suche `nearest_hit` testet jedes Objekt und merkt sich den nächsten Treffer. Erst für diesen wird die Normale aus den Eckpunktnormalen interpoliert. Die Suche `any_hit` dient dem

Schattentest und bricht beim ersten Treffer zwischen Punkt und Licht ab. Der Aufwand beider Suchen wächst damit mit dem Produkt aus der Zahl der Strahlen und der Zahl der Dreiecke. Der Renderer erzeugt je Pixel einen oder mehrere Sehstrahlen an zufälligen Stellen im Pixel und mittelt ihre Farben. Folgestrahlen werden bis zur Tiefe 5 verfolgt, ein Strahl ohne Treffer liefert einen hellblauen Himmel.

Der Wavefront-Import liest Eckpunkte, Normalen und Flächen in einem Durchlauf, zerlegt Polygone in Dreiecke und übernimmt die Eckpunktnormalen, sodass die Teekanne glatt erscheint. Kamera und Licht bestimmt das Programm aus dem Quader um alle Dreiecke. Die Kamera steht bei (0; 8,47; −12,02) auf der Seite mit negativem z. Sie blickt aus 30° Höhe mit 40° Öffnungswinkel auf die Mitte der Szene. Die Punktlichtquelle steht bei (1,36; 6,96; −2,71) über der Szene. Die 118 Unit-Tests mit GoogleTest liegen neben dem getesteten Code. Einer schickt 2000 Zufallsstrahlen in eine Szene aus 200 zufälligen Dreiecken und verlangt von beiden Schnittverfahren denselben Treffer im selben Abstand.

2.2 Schnittverfahren

Beide Verfahren bestimmen den Abstand des Treffers und zwei baryzentrische Koordinaten, mit denen die Normale interpoliert wird. Beide schneiden auch Rückseiten, da die Glastöpfe von innen getroffen werden. Das Verfahren nach Badouel [2] schneidet zuerst den Strahl mit der Ebene des Dreiecks. Normale und Abstand der Ebene hängen nur vom Dreieck ab und werden beim Anlegen einmal berechnet. Aus dem Abstand entlang des Strahls folgt der Schnittpunkt. Danach projiziert Badouel Punkt und Dreieck auf die Koordinatenebene, in der das Dreieck am größten erscheint. Dazu entfällt die Koordinate, in der die Normale ihre größte Komponente hat. In der Ebene wird der Punkt über zwei Kanten des Dreiecks ausgedrückt und das Gleichungssystem mit zwei Unbekannten gelöst (Listing 1). Steht die erste Kante senkrecht auf der ersten Achse, gilt dafür ein eigener Zweig. Nach der ersten Koordinate kann der Test abbrechen. Die frühere Fassung verwendete hier den Test aus dem Labor, der im Raum mit drei Kreuzprodukten prüft, ob der Punkt für jede Kante auf der Innenseite liegt. Das entspricht nicht dem Verfahren nach Badouel und wurde ersetzt.

```
float t = -(plane_d + normal * ray.origin) / (normal * ray.direction);
if (t < 0.f) return false;
Vector3df p = ray.origin + t * ray.direction;
float u0 = p[axis_u] - a[axis_u], v0 = p[axis_v] - a[axis_v];   // Punkt, b und c relativ zu a
float u1 = b[axis_u] - a[axis_u], v1 = b[axis_v] - a[axis_v];
float u2 = c[axis_u] - a[axis_u], v2 = c[axis_v] - a[axis_v];
if (u1 == 0.f) { beta = u0 / u2;                                  // b - a senkrecht zur ersten Achse
    if (beta < 0.f || beta > 1.f) return false;  alpha = (v0 - beta * v2) / v1; }
else { beta = (v0 * u1 - u0 * v1) / (v2 * u1 - u2 * v1);
    if (beta < 0.f || beta > 1.f) return false;  alpha = (u0 - beta * u2) / u1; }
if (alpha < 0.f || alpha + beta > 1.f) return false;
```

Listing 1: Schnitttest nach Badouel (`scene/scene_object.cc`, gekürzt)

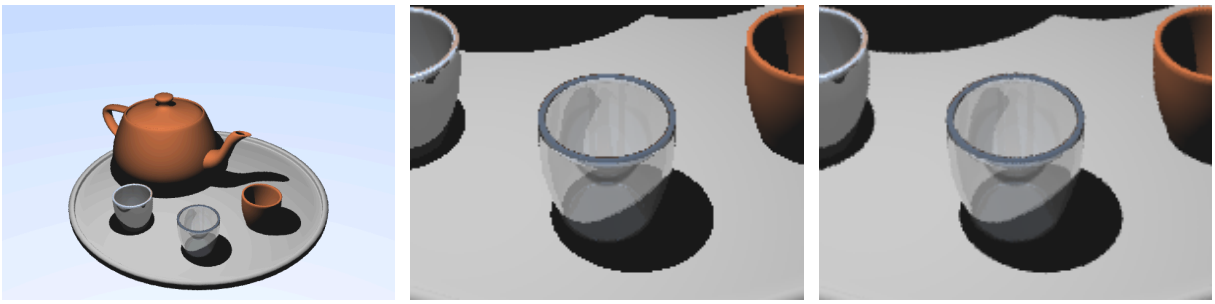
Das Verfahren nach Möller und Trumbore [3] beschreibt den Trefferpunkt direkt über zwei Kanten des Dreiecks und löst das entstehende Gleichungssystem mit der Cramerschen Regel. Ebene und Schnittpunkt werden nicht gebraucht und der Test kann schon nach der ersten und nach der zweiten baryzentrischen Koordinate abbrechen. Da bei der Brute-Force-Suche fast jeder Test sein Dreieck verfehlt, zählt vor allem der Weg bis zur ersten Abbruchprüfung. Möller-Trumbore braucht bis dahin 13 Multiplikationen und eine Division. Badouel braucht trotz der vorab berechneten Ebene ebenfalls 13 Multiplikationen, aber zwei Divisionen, von denen die zweite auf das Ergebnis der ersten wartet.

2.3 Beleuchtung und Materialien

Die Farbe in einem Trefferpunkt setzt sich nach Whitted [1] aus dem direkten Licht und zwei Folgestrahlen zusammen. Alle Gewichte stammen aus der Materialdatei. Die ambiente Farbe gilt überall. Für jede Lichtquelle, die der Schattenstrahl erreicht, kommen der diffuse Anteil nach Lambert und ein Glanzlicht nach Phong [4] hinzu. Die spiegelnde Farbe bestimmt die Stärke des Glanzlichts und der Exponent seine Schärfe. Dieselbe spiegelnde Farbe gewichtet auch den

gespiegelten Strahl. Die Deckkraft gibt laut MTL-Spezifikation an, wie stark die Oberfläche das Dahinterliegende überdeckt. Ambiente Farbe, diffuser Anteil und Glanzlicht zählen deshalb nur mit der Deckkraft, den Rest liefert der gebrochene Strahl. In der Testszene spiegeln damit der Porzellantopf und der Glastopf. Teekanne und Tontopf sowie die Platte aus Aluminium haben keine spiegelnde Farbe und bleiben matt. In einer früheren Fassung spiegelte nur ein Material mit dem Beleuchtungsmodell 3 oder 5., sodass die Testszene keine Spiegelung enthielt. Der Schattenstrahl startet leicht entlang der Normale versetzt. Seine Richtung ist der unnormierte Vektor zum Licht, sodass jeder Treffer vor dem Licht Schatten wirft. Zeigt die Fläche vom Licht weg, entfällt er ganz. Der gebrochene Strahl folgt dem Brechungsgesetz von Snellius. Ob er in das Glas eintritt oder es verlässt, entscheidet die Orientierung des Dreiecks. Bei Totalreflexion geht sein Gewicht an den gespiegelten Strahl. Glas wirft wie jedes andere Objekt einen vollen Schatten.

2.4 Bildergebnisse



(a) 640 × 480, 4 Sehstrahlen (b) Ausschnitt, 1 Sehstrahl (c) Ausschnitt, 4 Sehstrahlen

Abbildung 2: Testszene mit Möller-Trumbore und vergrößerter Ausschnitt um den Glastopf

Abbildung 2 zeigt die Testszene mit der festgelegten Kamera. Zu erkennen sind die harten Schatten auf der Platte, die Spiegelung im Porzellantopf und die Brechung im Glastopf, durch den Platte und Rand verzerrt erscheinen. Auf Porzellan und Glas liegt zudem ein Glanzlicht. Im vergrößerten Ausschnitt entstehen mit einem Sehstrahl je Pixel an den Rändern und Schattengrenzen Treppenstufen. Mit vier Strahlen an zufälligen Stellen im Pixel werden die Übergänge weicher. Die Bilder beider Schnittverfahren unterscheiden sich bei 160 × 120 Pixeln in 1 von 19 200 und bei 320 × 240 Pixeln in 7 von 76 800 Pixeln, jeweils um höchstens eine Stufe. Bei 640 × 480 Pixeln sind es 20 von 307 200, davon 18 um höchstens eine Stufe. Ursache sind Rundungsfehler. Ein Pixel am Rand der Platte weicht dabei stark ab. Dort läuft der Sehstrahl genau über die gemeinsame Kante zweier Dreiecke. Möller-Trumbore verfehlt durch Rundung beide Dreiecke und zeigt den Himmel, Badouel trifft eines davon. Keines der beiden Verfahren ist an gemeinsamen Kanten wasserdicht.

2.5 Zeitmessungen

Vergleich der Schnittverfahren. Tabelle I stellt beide Verfahren in der Referenzkonfiguration gegenüber, also bei 320 × 240 Pixeln, einem Sehstrahl je Pixel und Rekursionstiefe 5.

Tabelle I: Vergleich der Schnittverfahren (320 × 240 Pixel, 1 Sehstrahl je Pixel, 3 Läufe)

Verfahren	**Laden [ms]**	**Min [s]**	**Median [s]**	**Mittel [s]**	**je Pixel [µs]**	**je Test [ns]**
Badouel	25	38,74	38,79	38,80	505	16,77
Möller-Trumbore	23	18,65	19,54	19,37	254	8,45

Möller-Trumbore ist um den Faktor 1,99 schneller. Die Läufe streuen um höchstens 6,9 %, der Unterschied ist also kein Messrauschen. Beide Verfahren führen dieselben 2 312 686 972 Schnitttests aus. Ein Test dauert damit 8,45 ns gegenüber 16,77 ns. Bei gleicher Zahl an Multi-

plikationen erklärt vor allem die zweite Division den Abstand. Sie wartet auf den Schnittpunkt und dieser auf die erste Division, sodass der Prozessor die Rechnungen kaum überlappen kann. Zudem liest Badouel die Koordinaten über die gespeicherten Achsen einzeln aus dem Speicher und jedes Objekt ist 24 Byte größer. In einer früheren Fassung ohne virtuelle Methoden dauerte ein Test nach Möller-Trumbore 7,87 ns. Der Aufruf über die Tabelle der virtuellen Methoden und die einzeln angelegten Objekte kosten damit rund 7 %.

Strahlen und Schnitttests. Tabelle II schlüsselt die Strahlen der Referenzkonfiguration nach ihrer Art auf.

Tabelle II: Strahlen und Schnitttests bei 320 × 240 Pixeln (Möller-Trumbore)

Strahlart	**Strahlen**	**Schnitttests**	**Tests je Strahl**
Primärstrahlen	76 800	1 384 857 600	18 032
Folgestrahlen (Spiegelung, Brechung)	20 988	378 455 616	18 032
Schattenstrahlen	37 246	549 373 756	14 750
Summe	135 034	2 312 686 972	17 127

Primär- und Folgestrahlen prüfen alle 18 032 Dreiecke. Der Abbruch beim ersten Treffer spart wenig, ein Schattenstrahl prüft im Mittel 82 % der Dreiecke, da die meisten das Licht erreichen.

Skalierung mit Auflösung und Sehstrahlen je Pixel. Da jeder Strahl gleich viele Dreiecke prüft, sollte die Laufzeit proportional zur Zahl der Pixel und zur Zahl der Sehstrahlen je Pixel wachsen. Tabelle III vergleicht den gemessenen mit dem erwarteten Faktor.

Tabelle III: Laufzeit von Möller-Trumbore nach Auflösung und Sehstrahlen je Pixel (Median aus 3 Läufen)

Auflösung	**Sehstrahlen je Pixel**	**Laufzeit [s]**	**je Sehstrahl [µs]**	**Faktor gem.**	**Faktor erw.**
160 × 120	1	4,67	243,2	0,239	0,25
320 × 240	1	19,54	254,4	1,000	1,00
640 × 480	1	76,23	248,1	3,902	4,00
320 × 240	4	75,20	244,8	3,849	4,00
320 × 240	8	150,55	245,0	7,705	8,00
320 × 240	16	301,04	245,0	15,407	16,00

Die Laufzeit wächst wie erwartet linear. Keine Messung weicht um mehr als 3,8 % vom erwarteten Faktor ab. Badouel wurde zusätzlich bei allen drei Auflösungen gemessen und ist dort um den Faktor 1,94 bis 1,99 langsamer.

Folgerung. Der Aufwand des Brute-Force-Raytracers wächst mit dem Produkt aus der Zahl der Strahlen und der Zahl der Dreiecke. Hochgerechnet auf die voreingestellte Auflösung von 1280 × 720 Pixeln mit 16 Sehstrahlen je Pixel ergibt sich rund das 192-Fache der Referenzmessung, also etwa 63 Minuten für ein einziges Bild. Wesentlich schneller wird es nur, wenn ein Strahl deutlich weniger Dreiecke prüft. Dafür kann ein k-d-Baum hinter den beiden Suchen der Szene die Brute-Force-Suche ersetzen.

Literatur

[1] T. Whitted, „An Improved Illumination Model for Shaded Display“, *Communications of the ACM*, Bd. 23, Nr. 6, S. 343–349, 1980.

[2] D. Badouel, „An Efficient Ray-Polygon Intersection“, *Graphics Gems*. Academic Press, S. 390–393, 1990.

[3] T. Möller und B. Trumbore, „Fast, Minimum Storage Ray-Triangle Intersection“, *Journal of Graphics Tools*, Bd. 2, Nr. 1, S. 21–28, 1997.

[4] B. T. Phong, „Illumination for Computer Generated Pictures“, *Communications of the ACM*, Bd. 18, Nr. 6, S. 311–317, 1975.